

```
!pip install https://github.com/pandas-profiling/pandas-profiling/archive/master.zip
```

```
Collecting https://github.com/pandas-profiling/pandas-profiling/archive/master.zip
Using cached https://github.com/pandas-profiling/pandas-profiling/archive/master.zip
Preparing metadata (setup.py) ... done
Requirement already satisfied: scipy<1.12,>=1.4.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: pandas!=1.4.0,<2.1,>1.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: matplotlib<=3.7.3,>=3.2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: pydantic<2,>=1.8.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: PyYAML<6.1,>=5.0.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: Jinja2<3.2,>=2.11.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: visions[type_image_path]==0.7.5 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: numpy<1.26,>=1.16.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: htmlmin==0.1.12 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: phik<0.13,>=0.11.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: requests<3,>=2.24.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: tqdm<5,>=4.48.2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: seaborn<0.13,>=0.10.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: multimethod<2,>=1.4 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: statsmodels<1,>=0.13.2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: typeguard<5,>=4.1.2 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: imagehash==4.3.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: wordcloud>=1.9.1 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: dacite>=1.8 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0) (1.8.0)
Requirement already satisfied: numba<0.59.0,>=0.56.0 in /usr/local/lib/python3.10/dist-packages (from ydata-profiling==0.0.dev0)
Requirement already satisfied: PyWavelets in /usr/local/lib/python3.10/dist-packages (from imagehash==4.3.1->ydata-profiling==0.0.dev0)
Requirement already satisfied: pillow in /usr/local/lib/python3.10/dist-packages (from imagehash==4.3.1->ydata-profiling==0.0.dev0)
Requirement already satisfied: attrs>=19.3.0 in /usr/local/lib/python3.10/dist-packages (from visions[type_image_path]==0.7.5->ydata-profiling==0.0.dev0)
Requirement already satisfied: networkx>=2.4 in /usr/local/lib/python3.10/dist-packages (from visions[type_image_path]==0.7.5->ydata-profiling==0.0.dev0)
Requirement already satisfied: tangled-up-in-unicode>=0.0.4 in /usr/local/lib/python3.10/dist-packages (from visions[type_image_path]==0.7.5->ydata-profiling==0.0.dev0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (from Jinja2<3.2,>=2.11.1->ydata-profiling==0.0.dev0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<=3.7.3,>=3.2->ydata-profiling==0.0.dev0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib<=3.7.3,>=3.2->ydata-profiling==0.0.dev0)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib<=3.7.3,>=3.2->ydata-profiling==0.0.dev0)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<=3.7.3,>=3.2->ydata-profiling==0.0.dev0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib<=3.7.3,>=3.2->ydata-profiling==0.0.dev0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib<=3.7.3,>=3.2->ydata-profiling==0.0.dev0)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib<=3.7.3,>=3.2->ydata-profiling==0.0.dev0)
Requirement already satisfied: llvmlite<0.40,>=0.39.0dev0 in /usr/local/lib/python3.10/dist-packages (from numba<0.59.0,>=0.56.0->ydata-profiling==0.0.dev0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from numba<0.59.0,>=0.56.0->ydata-profiling==0.0.dev0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas!=1.4.0,<2.1,>1.1->ydata-profiling==0.0.dev0)
Requirement already satisfied: joblib>=0.14.1 in /usr/local/lib/python3.10/dist-packages (from phik<0.13,>=0.11.1->ydata-profiling==0.0.dev0)
Requirement already satisfied: typing-extensions>=4.2.0 in /usr/local/lib/python3.10/dist-packages (from pydantic<2,>=1.8.1->ydata-profiling==0.0.dev0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling==0.0.dev0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling==0.0.dev0)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling==0.0.dev0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-packages (from requests<3,>=2.24.0->ydata-profiling==0.0.dev0)
Requirement already satisfied: patsy>=0.5.2 in /usr/local/lib/python3.10/dist-packages (from statsmodels<1,>=0.13.2->ydata-profiling==0.0.dev0)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages (from patsy>=0.5.2->statsmodels<1,>=0.13.2->ydata-profiling==0.0.dev0)
```

```
from pandas_profiling import ProfileReport
%matplotlib inline
```

```
import pandas as pd
import math
import numpy as np
from datetime import datetime,date,time
from pandas import Timestamp
#import statsmodels.api as sm
```

```
import seaborn as sns
import matplotlib.pyplot as plt
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
```

▼ BLIND SPOT

Predicts whether you will encounter any blind spot while on your journey.

(Blind Spots are areas on road (known or unknown) due to which accidents may occur).

```
data = pd.read_csv('Blind_Spot_new.csv')
data = pd.DataFrame(data)
data.head()
```

	Row	Place	Area_Name	Weather_Condition	Blind_Spot	Accident_Prone	Label
0	1	Lonavala	Western Ghats	Clear	Yes	Yes	1
1	2	Raigad	Not Mentioned	Rainy	No	Yes	0
2	3	Pune	Western Ghats	Clear	Yes	Yes	1
3	4	Solapur	Not Mentioned	Foggy	Yes	Yes	1
4	5	Gulbarga	Not Mentioned	Clear	No	Yes	0

```
# Pandas profiling report
profile = ProfileReport(data, title='Blind_spot Data', explorative=True)
```

```
# export analysis results to an html page
profile.to_file('Blind_spot Data.html')
```

```
Summarize dataset: 100% 17/17 [00:02<00:00, 3.49it/s, Completed]
Generate report structure: 100% 1/1 [00:08<00:00, 8.36s/it]
Render HTML: 100% 1/1 [00:01<00:00, 1.37s/it]
Export report to file: 100% 1/1 [00:00<00:00, 13.38it/s]
```

```
df_new = data.copy()
```

```
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor
```

```
# Prepare the data
X = df_new.drop(columns=['Row', 'Label'], axis=1)
y = df_new['Label']
```

```
# Encode categorical variables
label_encoders = {}
categorical_cols = ['Place', 'Area_Name', 'Weather_Condition', 'Blind_Spot', 'Accident_Prone']
for col in categorical_cols:
    label_encoders[col] = LabelEncoder()
    X[col] = label_encoders[col].fit_transform(X[col])

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Create a RandomForestClassifier model
from sklearn.ensemble import RandomForestRegressor
model_blind = RandomForestRegressor(n_estimators=100, random_state=42)

# Train the model
model_blind.fit(X_train, y_train)
```

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

```
# Make predictions
y_pred = model_blind.predict(X_test)
```

```
from sklearn.metrics import r2_score
r_squared = r2_score(y_test, y_pred)
N = len(y_test)
k = X_test.shape[1]

# Calculate adjusted R-squared
adjusted_r_squared = 1 - (1 - r_squared) * (N - 1) / (N - k - 1)
print("Adjusted R-squared:", adjusted_r_squared)
```

```
Adjusted R-squared: 1.0
```

```
# Reverse the label encoding for categorical columns in the new DataFrame
new_data = {
    'Place': ['Pune'],
```

```

    'Area_Name': ['Western Ghats'],
    'Weather_Condition': ['Clear'],
    'Blind_Spot': ['Yes'],
    'Accident_Prone': ['Yes']
}

# Create a new DataFrame with the reversed categorical values
new_df_blind_reversed = pd.DataFrame(new_data)

# Reverse the label encoding for categorical columns in the new DataFrame
for col in categorical_cols:
    if new_df_blind_reversed[col][0] in label_encoders[col].classes_:
        new_df_blind_reversed[col] = label_encoders[col].transform([new_df_blind_reversed[col]])[0]
    else:
        # Handle unseen labels gracefully by using a default value
        new_df_blind_reversed[col] = -1 # Use -1 to represent unseen labels

# Make predictions using the trained model
current_blindspot_prediction = model_blind.predict(new_df_blind_reversed)

area_name = new_data['Area_Name'][0]

print(f"Prediction of blind spot around {area_name}:") # 0 means no blind spot currently present & 1 means a blind spot pr

if 0 <= current_blindspot_prediction[0] <= 0.8:
    print('No blind spot present')
else:
    print('Blind spot is present')
print()
print()

```

Prediction of blind spot around Western Ghats:
Blind spot is present

```

/usr/local/lib/python3.10/dist-packages/sklearn/preprocessing/_label.py:134: DataConversionWarning: A column-vector y was passed as a 1D array, which will be deprecated in the future. Use y = column_or_id(y, dtype=self.classes_.dtype, warn=True)
/usr/local/lib/python3.10/dist-packages/sklearn/preprocessing/_label.py:134: DataConversionWarning: A column-vector y was passed as a 1D array, which will be deprecated in the future. Use y = column_or_id(y, dtype=self.classes_.dtype, warn=True)
/usr/local/lib/python3.10/dist-packages/sklearn/preprocessing/_label.py:134: DataConversionWarning: A column-vector y was passed as a 1D array, which will be deprecated in the future. Use y = column_or_id(y, dtype=self.classes_.dtype, warn=True)
/usr/local/lib/python3.10/dist-packages/sklearn/preprocessing/_label.py:134: DataConversionWarning: A column-vector y was passed as a 1D array, which will be deprecated in the future. Use y = column_or_id(y, dtype=self.classes_.dtype, warn=True)
/usr/local/lib/python3.10/dist-packages/sklearn/preprocessing/_label.py:134: DataConversionWarning: A column-vector y was passed as a 1D array, which will be deprecated in the future. Use y = column_or_id(y, dtype=self.classes_.dtype, warn=True)
/usr/local/lib/python3.10/dist-packages/sklearn/preprocessing/_label.py:134: DataConversionWarning: A column-vector y was passed as a 1D array, which will be deprecated in the future. Use y = column_or_id(y, dtype=self.classes_.dtype, warn=True)

```

✓ Air Quality and Weather Forecasting

Informs about pollution level today and weather condition.

```
df_air = pd.read_csv('Air_Quality.csv')
```

```

# Convert the 'date_column' to datetime format
df_air['Date'] = pd.to_datetime(df_air['Date'])
df_air.head()

```

```

<ipython-input-17-c74e0a004f5a>:4: UserWarning: Parsing dates in DD/MM/YYYY format when dayfirst=False (the default) was specified
df_air['Date'] = pd.to_datetime(df_air['Date'])

```

	Index	Date	Location	PM10 (µg/m³)	CO (ppm)	NO2 (ppb)	SO2 (ppb)	O3 (ppb)	Temperature (°C)	Humidity (%)	Weather_Condition	PM2.5
0	1	2023-01-01	Mumbai	20	0.2	10	5	35	22.5	50	Clear	15.0
1	2	2023-01-01	Pune	22	0.3	12	6	38	22.7	51	Rainy	18.0
2	3	2023-01-01	Nagpur	25	0.4	15	7	40	22.8	52	Clear	22.0
3	4	2023-01-01	Thane	28	0.5	18	8	42	23.0	53	Foggy	25.0

```

# Pandas profiling report
profile = ProfileReport(df_air, title='Air_Quality Data', explorative=True)

# export analysis results to an html page
profile.to_file('Air_Quality Data.html')

```

```

Summarize dataset: 100%      85/85 [00:30<00:00, 3.09it/s, Completed]
Generate report structure: 100%      1/1 [00:08<00:00, 8.79s/it]
Render HTML: 100%      1/1 [00:02<00:00, 2.17s/it]
Export report to file: 100%      1/1 [00:00<00:00, 14.60it/s]

```

```

from scipy import stats
from scipy.stats import boxcox

```

```

numerical_columns = ['PM10 (µg/m³)', 'NO2 (ppb)', 'SO2 (ppb)', 'O3 (ppb)']
new_df = df_air.copy()

```

```

for column in numerical_columns:
    # Add a small constant to avoid issues with zero or negative values
    transformed_data, lambda_bestfit = stats.boxcox(new_df[column] + 0.01)
    new_df[column] = transformed_data

```

```

from scipy.stats import skew

```

```

# Separate numerical and categorical columns
numerical_columns = new_df.select_dtypes(include=[np.number]).columns
categorical_columns = new_df.select_dtypes(exclude=[np.number]).columns

```

```

# Initialize counters
normal_skewness_count = 0
positive_skewness_count = 0
negative_skewness_count = 0

```

```

# Iterate through numerical columns and calculate skewness
for column in numerical_columns:
    skew_value = skew(new_df[column])
    if -1 <= skew_value <= 1:
        normal_skewness_count += 1
    elif skew_value > 1:
        positive_skewness_count += 1
    elif skew_value < -1:
        negative_skewness_count += 1

```

```

# Print the counts for each skewness category
print(f"Columns with normal skewness: {normal_skewness_count}")
print(f"Columns with positive skewness: {positive_skewness_count}")
print(f"Columns with negative skewness: {negative_skewness_count}")

```

```

Columns with normal skewness: 9
Columns with positive skewness: 0
Columns with negative skewness: 0

```

```

from sklearn.preprocessing import StandardScaler, LabelEncoder

```

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor

```

```

new_df['Weather_Condition'].replace({'Clear': 0, 'Foggy': 0.5, 'Rainy': 1}, inplace=True)
new_df.head()

```

	Index	Date	Location	PM10 (µg/m³)	CO (ppm)	NO2 (ppb)	SO2 (ppb)	O3 (ppb)	Temperature (°C)	Humidity (%)	Weather_Condition	PM2.5
0	1	2023-01-01	Mumbai	3.484739	0.2	2.493037	1.711262	3.263861	22.5	50	0.0	15.0
1	2	2023-01-01	Pune	3.613272	0.3	2.707272	1.917639	3.332825	22.7	51	1.0	18.0
2	3	2023-01-01	Nagpur	3.787576	0.4	2.973143	2.094356	3.375699	22.8	52	0.0	22.0

```

# Prepare the data
X = new_df.drop(columns=['Index', 'Date', 'Weather_Condition', 'PM2.5'], axis=1)
y_1 = new_df['PM2.5']
y_2 = new_df['Weather_Condition']

```

```

# Encode categorical variables
label_encoders = {}

```

```
categorical_cols = ['Location', 'CO (ppm)']
for col in categorical_cols:
    label_encoders[col] = LabelEncoder()
    X[col] = label_encoders[col].fit_transform(X[col])
```

```
# Split the data into training and testing sets
X_train_1, X_test_1, y_train_1, y_test_1 = train_test_split(X, y_1, test_size=0.2, random_state=42)

# Create a RandomForestClassifier model
from sklearn.ensemble import RandomForestRegressor
model_1 = RandomForestRegressor(n_estimators=100, random_state=42)

model_1.fit(X_train_1, y_train_1)
```

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

```
from sklearn.model_selection import GridSearchCV

# Define the hyperparameters and their possible values to search
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
```

```
# Create the GridSearchCV object
grid_search = GridSearchCV(estimator=model_1, param_grid=param_grid, cv=5, n_jobs=-1, verbose=2)

# Perform the grid search on your data
grid_search.fit(X_train_1, y_train_1)
```

Fitting 5 folds for each of 81 candidates, totalling 405 fits

```
GridSearchCV
  estimator: RandomForestRegressor
    RandomForestRegressor
```

```
# Get the best parameters and the best estimator
best_params_1 = grid_search.best_params_
best_estimator_1 = grid_search.best_estimator_

print("Best Hyperparameters:", best_params_1)
print("Best Estimator:", best_estimator_1)
```

```
Best Hyperparameters: {'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 2, 'n_estimators': 200}
Best Estimator: RandomForestRegressor(max_depth=10, n_estimators=200, random_state=42)
```

```
y_pred_1 = best_estimator_1.predict(X_test_1)
```

```
from sklearn.metrics import r2_score
r_squared = r2_score(y_test_1, y_pred_1)
N = len(y_test_1)
k = X_test_1.shape[1]

# Calculate adjusted R-squared
adjusted_r_squared = 1 - (1 - r_squared) * (N - 1) / (N - k - 1)
print("Adjusted R-squared:", adjusted_r_squared)
```

```
Adjusted R-squared: 0.9999661218061213
```

```
# Create a new DataFrame with the features for prediction
new_data = {
    'Location': [2],
    'PM10 (µg/m³)': [3.6],
    'CO (ppm)': [2],
    'NO2 (ppb)': [2.7],
    'SO2 (ppb)': [1.9],
    'O3 (ppb)': [4.2],
    'Temperature (°C)': [-1],
    'Humidity (%)': [50],
}

new_df_air = pd.DataFrame(new_data)
```

```
# Make predictions using the trained model
pollution_prediction = best_estimator_1.predict(new_df_air)

print("Predicted PM2.5 Level:", pollution_prediction[0])
```

Predicted PM2.5 Level: 59.94624015286848

```
# Split the data into training and testing sets
X_train_2, X_test_2, y_train_2, y_test_2 = train_test_split(X, y_2, test_size=0.2, random_state=42)
```

```
# Create a RandomForestClassifier model
from sklearn.ensemble import RandomForestRegressor
model = RandomForestRegressor(n_estimators=100, random_state=42)

model.fit(X_train_2, y_train_2)
```

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

```
from sklearn.model_selection import GridSearchCV

# Define the hyperparameters and their possible values to search
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
```

```
# Create the GridSearchCV object
grid_search = GridSearchCV(estimator=model_1, param_grid=param_grid, cv=5, n_jobs=-1, verbose=2)

# Perform the grid search on your data
grid_search.fit(X_train_2, y_train_2)
```

Fitting 5 folds for each of 81 candidates, totalling 405 fits

```
GridSearchCV
  estimator: RandomForestRegressor
    RandomForestRegressor
```

```
# Get the best parameters and the best estimator
best_params_2 = grid_search.best_params_
best_estimator_2 = grid_search.best_estimator_

print("Best Hyperparameters_2:", best_params_2)
print("Best Estimator_2:", best_estimator_2)
```

Best Hyperparameters_2: {'max_depth': 10, 'min_samples_leaf': 4, 'min_samples_split': 10, 'n_estimators': 200}
 Best Estimator_2: RandomForestRegressor(max_depth=10, min_samples_leaf=4, min_samples_split=10, n_estimators=200, random_state=42)

```
# Make predictions
y_pred_2 = best_estimator_2.predict(X_test_2)
```

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_test_2, y_pred_2)
print("Mean Absolute Error (MAE):", mae)
```

Mean Absolute Error (MAE): 0.399081217557247

```
# Create a new DataFrame with the features for prediction
new_data_2 = {
    'Location': [2],
    'PM10 (µg/m³)': [3.6],
    'CO (ppm)': [2],
    'NO2 (ppb)': [2.7],
    'SO2 (ppb)': [1.9],
    'O3 (ppb)': [4.2],
    'Temperature (°C)': [-1],
    'Humidity (%)': [50],
}

new_df_weather = pd.DataFrame(new_data_2)
```

```
# Make predictions using the trained model
weather_condition = best_estimator_2.predict(new_df_weather)

print("Predicted Weather Condition:") #Between 0 to 0.5 i.e between clear and rainy

if(0<=weather_condition[0]<0.5):
    print('Clear')
elif(0.5<=weather_condition[0]<0.8):
    print('Foggy')
else:
    print('Rainy')
```

Predicted Weather Condition:
Clear

▼ Traffic Volume & Road Recommendation System

1. Tells about next day traffic level.
2. Collects feedback from various factors to recommend roads that need to develop. (Government level)
3. Predicts if alternative waterway transport development is possible so as to reduce dependency on roadway.(Inspired by Ghati Shakti Bharat scheme)

```
df_t = pd.read_csv('Traffic_Recommend.csv')

# Convert the 'date_column' to datetime format
df_t['Date'] = pd.to_datetime(df_t['Date'])
df_t.head()
```

<ipython-input-44-64ea568cc32c>:4: UserWarning: Parsing dates in DD/MM/YYYY format when dayfirst=False (the default) was spe
df_t['Date'] = pd.to_datetime(df_t['Date'])

	Index	Date	Temperature (Celsius)	Humidity (%)	Wind Speed (km/h)	Pollution Level (PM2.5)	Road_ID	Congestion Level	Safety Rating	Air Quality Rating
0	1	2023-01-01	25	50	5	15	1	0.1	4.5	3.8
1	2	2023-02-01	28	55	8	18	2	0.2	4.8	4.1
2	3	2023-03-01	30	58	6	20	3	0.4	4.2	3.9
3	4	2023-04-01	22	45	4	12	4	0.3	4.0	4.2
4	5	2023-05-01	24	48	7	14	5	0.2	4.3	4.0

```
df_t.drop('Index', axis=1, inplace=True)
df_t.head()
```

	Date	Temperature (Celsius)	Humidity (%)	Wind Speed (km/h)	Pollution Level (PM2.5)	Road_ID	Congestion Level	Safety Rating	Air Quality Rating
0	2023-01-01	25	50	5	15	1	0.1	4.5	3.8
1	2023-02-01	28	55	8	18	2	0.2	4.8	4.1
2	2023-03-01	30	58	6	20	3	0.4	4.2	3.9
3	2023-04-01	22	45	4	12	4	0.3	4.0	4.2
4	2023-05-01	24	48	7	14	5	0.2	4.3	4.0

```
# Convert 'Date' to datetime and set it as the index
df_t['Date'] = pd.to_datetime(df_t['Date'])
df_t.set_index('Date', inplace=True)
```

```
# Pandas profiling report
profile = ProfileReport(df_t, title='Transport and Recommend Data', explorative=True)

# export analysis results to an html page
profile.to_file('Transport and Recommend Data.html')
```

Summarize dataset: 100%	66/66 [00:17<00:00, 2.71it/s, Completed]
Generate report structure: 100%	1/1 [00:05<00:00, 5.36s/it]
Render HTML: 100%	1/1 [00:01<00:00, 1.65s/it]
Export report to file: 100%	1/1 [00:00<00:00, 16.81it/s]

```
df = df_t.copy()
```

```
X = df.drop('Congestion Level', axis=1)
y = df['Congestion Level']
```

```
from sklearn.linear_model import LinearRegression
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
LinearRegression()
LinearRegression()
```

```
# Make predictions
y_pred = model.predict(X_test)
```

```
from sklearn.metrics import r2_score
r_squared = r2_score(y_test, y_pred)
N = len(y_test)
k = X_test.shape[1]

# Calculate adjusted R-squared
adjusted_r_squared = 1 - (1 - r_squared) * (N - 1) / (N - k - 1)
print("Adjusted R-squared:", adjusted_r_squared)
```

Adjusted R-squared: 0.9983557156667826

```
next_day_data = pd.DataFrame({
    'Temperature (Celsius)': [27],
    'Humidity (%)': [53],
    'Wind Speed (km/h)': [6],
    'Pollution Level (PM2.5)': [16],
    'Road_ID': [1],
    'Safety Rating': [4.2],
    'Air Quality Rating': [4.3]
})
```

```
# Predict congestion level for the next day
predicted_congestion = model.predict(next_day_data)
```

```
print("Predicted Traffic Volume for Tomorrow:", predicted_congestion[0])
```

```
if(0<=predicted_congestion[0]<0.4):
    print('No Traffic')
elif(0.4<=predicted_congestion[0]<0.7):
    print('Traffic')
else:
    print('High Traffic')
```

Predicted Traffic Volume for Tomorrow: 0.24055946763920127
No Traffic

Waterway possibility

```
df_water = pd.read_csv('Water_Transport_Recommend.csv')
df_water.head()
```

	Index	Location	Population	Economic Activity	Distance to Major Ports	Terrain Type	Historical Waterway Usage	Feasibility
0	1	Mumbai	1,24,78,447	High	20 miles	Coastal	Moderate	Yes
1	2	Bangalore	84,43,675	Medium	180 miles	Hilly	Low	No
2	3	Chennai	70,88,222	High	30 miles	Coastal	High	Yes
3	4	Delhi	1,10,34,555	High	150 miles	Flat	Low	No
4	5	Kolkata	44,96,694	Medium	40 miles	Coastal	Moderate	Yes

```
df = df_water.copy()
from sklearn.preprocessing import StandardScaler, LabelEncoder
```

```
def extract_numerical_distance(text):
    return int(text.split()[0])

df["Distance to Major Ports"] = df["Distance to Major Ports"].apply(extract_numerical_distance)
df["Location"] = pd.Categorical(df["Location"])
df.head()
```

	Index	Location	Population	Economic Activity	Distance to Major Ports	Terrain Type	Historical Waterway Usage	Feasibility
0	1	Mumbai	1,24,78,447	High	20	Coastal	Moderate	Yes
1	2	Bangalore	84,43,675	Medium	180	Hilly	Low	No
2	3	Chennai	70,88,222	High	30	Coastal	High	Yes
3	4	Delhi	1,10,34,555	High	150	Flat	Low	No
4	5	Kolkata	44,96,694	Medium	40	Coastal	Moderate	Yes

```
# Pandas profiling report
profile = ProfileReport(df, title='Water_Transport_Recommend Data', explorative=True)

# export analysis results to an html page
profile.to_file('Water_Transport_Recommend Data.html')
```

```
Summarize dataset: 100%                21/21 [00:02<00:00, 4.01it/s, Completed]
Generate report structure: 100%         1/1 [00:05<00:00, 5.81s/it]
Render HTML: 100%                      1/1 [00:00<00:00, 2.41it/s]
Export report to file: 100%             1/1 [00:00<00:00, 32.15it/s]
```

```
df['Feasibility'].replace({'Yes': 1, 'No': 0}, inplace=True)
df.head()
```

	Index	Location	Population	Economic Activity	Distance to Major Ports	Terrain Type	Historical Waterway Usage	Feasibility
0	1	Mumbai	1,24,78,447	High	20	Coastal	Moderate	1
1	2	Bangalore	84,43,675	Medium	180	Hilly	Low	0
2	3	Chennai	70,88,222	High	30	Coastal	High	1
3	4	Delhi	1,10,34,555	High	150	Flat	Low	0
4	5	Kolkata	44,96,694	Medium	40	Coastal	Moderate	1

```
X = df.drop(['Index', 'Feasibility'], axis=1)
y = df['Feasibility']
```

```
# Encode categorical variables
label_encoders = {}
categorical_cols = ['Location', 'Population', 'Economic Activity', 'Terrain Type', 'Historical Waterway Usage']
for col in categorical_cols:
    label_encoders[col] = LabelEncoder()
    X[col] = label_encoders[col].fit_transform(X[col])
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

```
y_pred = model.predict(X_test)
```

```
from sklearn.metrics import r2_score
r_squared = r2_score(y_test, y_pred)
N = len(y_test)
k = X_test.shape[1]

# Calculate adjusted R-squared
adjusted_r_squared = 1 - (1 - r_squared) * (N - 1) / (N - k - 1)
print("Adjusted R-squared:", adjusted_r_squared)
```

Adjusted R-squared: 0.979

X_train

	Location	Population	Economic Activity	Distance to Major Ports	Terrain Type	Historical Waterway Usage
12	18	22	2	80	1	1
4	25	37	2	40	0	2
37	13	44	1	60	1	1
8	36	30	0	50	2	2
3	9	0	0	150	1	1
6	20	29	1	100	1	1
41	15	49	2	10	0	0
46	27	36	1	20	0	1
47	38	27	2	30	0	2
15	49	19	0	10	0	0
9	28	26	2	100	1	1
16	35	18	2	60	1	1
24	22	7	1	180	2	1
34	40	23	1	100	2	1
31	48	14	2	10	0	0
0	30	1	0	20	0	2
44	29	32	2	60	1	2
27	1	11	1	60	1	1
33	11	40	2	30	1	2
5	17	45	2	80	1	1
29	19	12	2	60	1	2
11	32	24	2	40	2	2
36	12	35	2	100	1	1
1	4	48	2	180	2	1
21	33	4	0	20	0	0
2	7	46	0	30	0	0
43	24	42	0	40	0	0
35	45	33	2	40	0	1
23	14	47	0	40	1	1
40	21	38	1	120	2	1
10	23	25	1	60	1	1

```
next_data = pd.DataFrame({
    'Location': [47],
    'Population': [56],
    'Economic Activity': [2],
    'Distance to Major Ports': [20],
    'Terrain Type': [2],
    'Historical Waterway Usage': [1]
})
```

```
# Predict congestion level for the next day
predicted_waterway = model.predict(next_data)

print("Is waterway development possible:")
if(0<=predicted_waterway[0]<0.8):
    print('Yes')
else:
    print('No')
```

```
Is waterway development possible:
Yes
```

Recommendation System

```
# Define weights for different factors
weights = {
    'Congestion Level': 0.4,
    'Safety Rating': 0.4,
    'Air Quality Rating': 0.2,
}
```

```
# Calculate a recommendation score for each road
df_t['Recommendation_Score'] = (
    weights['Congestion Level'] * df_t['Congestion Level'] +
    weights['Safety Rating'] * df_t['Safety Rating'] +
    weights['Air Quality Rating'] * df_t['Air Quality Rating']
)

# Sort roads by recommendation score in descending order
df_t = df_t.sort_values(by='Recommendation_Score', ascending=False)
```

```
df_t.head()
```

	Temperature (Celsius)	Humidity (%)	Wind Speed (km/h)	Pollution Level (PM2.5)	Road_ID	Congestion Level	Safety Rating	Air Quality Rating	Recommendation_Score
Date									
2024-10-02	359	381	6	320	1	3.72	4.8	4.7	4.348
2024-09-02	358	380	4	319	5	3.71	4.7	4.8	4.324
2024-01-31	349	371	5	310	1	3.62	4.8	4.7	4.308
2024-11-02	360	382	7	321	2	3.73	4.7	4.6	4.292
2024-01-30	348	370	6	309	5	3.61	4.7	4.8	4.284

```
# Display the top recommended road improvements
top_recommendations = df_t[['Road_ID', 'Recommendation_Score']].head()
print("Top Recommended Road Improvements:")
print(top_recommendations)
```

```
Top Recommended Road Improvements:
      Road_ID  Recommendation_Score
Date
2024-10-02      1             4.348
2024-09-02      5             4.324
2024-01-31      1             4.308
2024-11-02      2             4.292
```